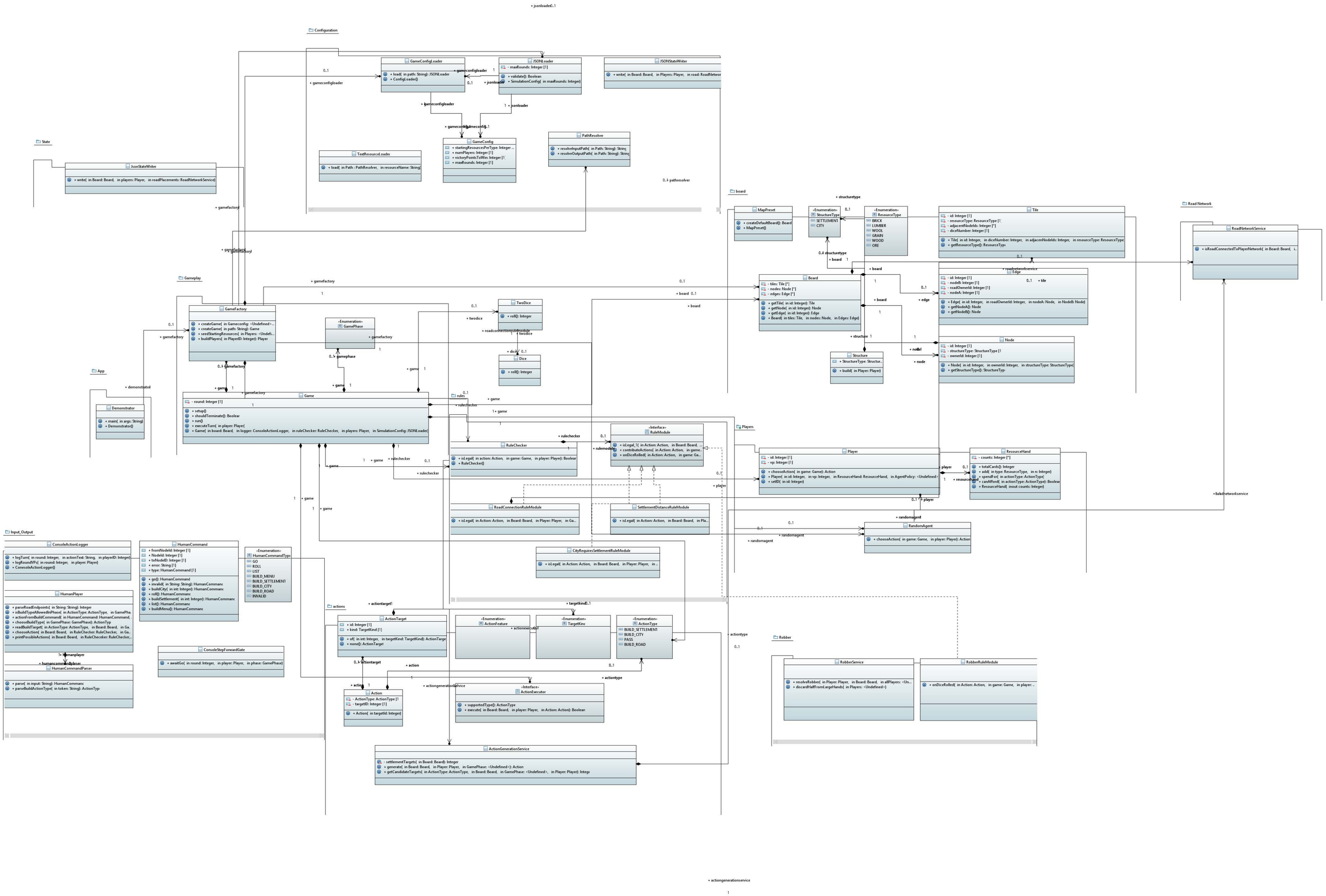
Configuration
GameConfigLoader
+ load(in path: String): JSONLoader
+ ConfigLoader()
JSONLoader
- maxRounds: Integer [1]
+ validate(): Boolean
+ SimulationConfig(in maxRounds: Integer)
JSONStateWriter
+ write(in Board: Board, in Players: Player, in road: RoadNetwor
TextResourceLoader
+ load(in Path : PathResolver, in resourceName: String)
GameConfig
+ startingResourcesPerType: Integer ...
+ numPlayers: Integer [1]
+ victoryPointsToWin: Integer [1]
+ maxRounds: Integer [1]
PathResolver
+ resolveInputPath(in Path: String): String
+ resolveOutputPath(in Path: String): String
State
JsonStateWriter
+ write(in Board: Board, in players: Player, in roadPlacements: RoadNetworkService)
board
MapPreset
+ createDefaultBoard(): Board
+ MapPreset()
«Enumeration»
StructureType
SETTLEMENT
CITY
«Enumeration»
ResourceType
BRICK
LUMBER
WOOL
GRAIN
WOOD
ORE
Tile
- id: Integer [1]
- resourceType: ResourceType [1]
- adjacentNodeIds: Integer [*]
- diceNumber: Integer [1]
+ Tile(in id: Integer, in diceNumber: Integer, in adjacenNodeIds: Integer, in resourceType: ResourceType
+ getResourceType(): ResourceTyp
Road Network
RoadNetworkService
+ isRoadConnectedToPlayerNetwork(in Board: Board, i...
Board
- tiles: Tile [*]
- nodes: Node [*]
- edges: Edge [*]
+ getTile(in id: Integer): Tile
+ getNode(in id: Integer): Node
+ getEdge(in id: Integer): Edge
+ Board(in tiles: Tile, in nodes: Node, in Edges: Edge)
Edge
- id: Integer [1]
- nodeB: Integer [1]
- roadOwnerId: Integer [1]
- nodeA: Integer [1]
+ Edge(in id: Integer, in roadOwnerId: Integer, in nodeA: Node, in NodeB: Node)
+ getNodeA(): Node
+ getNodeB(): Node
Structure
+ StructureType: Structur...
+ build(in Player: Player)
Node
- id: Integer [1]
- structureType: StructureType [1
- ownerId: Integer [1]
+ Node(in id: Integer, in ownerId: Integer, in structureType: StructureType
+ getStructureType(): StructureTyp
Gameplay
GameFactory
+ createGame(in Gameconfig: <Undefined>...
+ createGame(in path: String): Game
+ seedStartingResources(in Players: <Undefi...
+ buildPlayers(in PlayerID: Integer): Player
«Enumeration»
GamePhase
TwoDice
+ roll(): Integer
Dice
+ roll(): Integer
App
Demonstrator
+ main(in args: String)
+ Demonstrator()
Game
- round: Integer [1]
+ setup()
+ shouldTerminate(): Boolear
+ run()
+ executeTurn(in player: Player(
+ Game(in board: Board, in logger: ConsoleActionLogger, in ruleChecker: RuleChecker, in players: Player, in SimulationConfig: JSONLoader(
rules
RuleChecker
+ isLegal(in action: Action, in game: Game, in player: Player): Boolear
+ RuleChecker()
«Interface»
RuleModule
+ isLegal_1(in Action: Action, in Board: Board, ...
+ contributeActions(in Action: Action, in game...
+ onDiceRolled(in Action: Action, in game: Ga...
RoadConnectionRuleModule
+ isLegal(in Action: Action, in Board: Board, in Player: Player, in Ga...
SettlementDistanceRuleModule
+ isLegal(in Action: Action, in Board: Board, in Pla...
CityRequiresSettlementRuleModule
+ isLegal(in Action: Action, in Board: Board, in Player: Player, in ...
Players
Player
- id: Integer [1]
- vp: Integer [1]
+ chooseAction(in game: Game): Action
+ Player(in id: Integer, in vp: Integer, in ResourceHand: ResourceHand, in AgentPolicy: <Undefined>)
+ setID(in id: Integer)
ResourceHand
- counts: Integer [*]
+ totalCards(): Integer
+ add(in type: ResourceType, in n: Integer)
+ spendFor(in actionType: ActionType)
+ canAfford(in actionType: ActionType): Boolear
+ ResourceHand(inout counts: Integer)
RandomAgent
+ chooseAction(in game: Game, in player: Player): Action
Input_Output
ConsoleActionLogger
+ logTurn(in round: Integer, in actionText: String, in playerID: Integer)
+ logRoundVPs(in round: Integer, in player: Player)
+ ConsoleActionLogger()
HumanCommand
+ fromNodeId: Integer [1]
+ NodeId: Integer [1]
+ toNodeID: Integer [1]
+ error: String [1]
+ type: HumanCommand [1]
+ go(): HumanCommand
+ invalid(in String: String): HumanCommanc
+ buildCity(in int: Integer): HumanCommanc
+ roll(): HumanCommanc
+ buildSettlement(in int: Integer): HumanCommanc
+ list(): HumanCommanc
+ buildMenu(): HumanCommanc
«Enumeration»
HumanCommandType
GO
ROLL
LIST
BUILD_MENU
BUILD_SETTLEMENT
BUILD_CITY
BUILD_ROAD
INVALID
HumanPlayer
+ parseRoadEndpoints(in String: String): Integer
+ isBuildTypeAllowedInPhase(in ActionType: ActionType, in GamePha
+ actionFromBuildCommand(in HumanCommand: HumanCommand,
+ chooseBuildType(in GamePhase: GamePhase): ActionTyp
+ readBuildTarget(in ActionType: ActionType, in Board: Board, in Ga
+ chooseAction(in Board: Board, in RuleChecker: RuleChecker, in Ga...
+ printPossibleActions(in Board: Board, in RuleChecker: RuleChecker,...
HumanCommandParser
+ parse(in input: String): HumanCommanc
+ parseBuildActionType(in token: String): ActionTyp
ConsoleStepForwardGate
+ awaitGo(in round: Integer, in player: Player, in phase: GamePhase)
actions
ActionTarget
+ id: Integer [1]
+ kind: TargetKind [1]
+ of(in int: Integer, in targetKind: TargetKind): ActionTarge
+ none(): ActionTarget
«Enumeration»
ActionFeature
«Enumeration»
TargetKinc
«Enumeration»
ActionType
BUILD_SETTLEMENT
BUILD_CITY
PASS
BUILD_ROAD
Action
- ActionType: ActionType [1
- targetID: Integer [1]
+ Action(in targetId: Integer)
«Interface»
ActionExecutor
+ supportedType(): ActionType
+ execute(in Board: Board, in player: Player, in Action: Action): Boolean
ActionGenerationService
- settlementTargets(in Board: Board): Integer
+ generate(in Board: Board, in Player: Player, in GamePhase: <Undefined>): Action
+ getCandidateTargets(in ActionType: ActionType, in Board: Board, in GamePhase: <Undefined>, in Player: Player): Intege
Robber
RobberService
+ resolveRobber(in Player: Player, in Board: Board, in allPlayers: <Un...
+ discardHalfFromLargeHands(in Players: <Undefined>)
RobberRuleModule
+ onDiceRolled(in Action: Action, in game: Game, in player: ...